

Programação em Python

Unidade 1

Desenvolver Algoritmos Utilizando Lógica de Programação

Sessão 06

Pyhton

Funções

- É um dos tipos de estruturas de código chamadas de Módulos.
- Existem dois tipos de módulos:
 - Procedimento (ou subrotina, subprograma).
 - Função.
- São executadas apenas quando chamadas.
- Permitem a passagem de valores do módulo chamador para o módulo chamado.
- Os valores passados são chamados de *parâmetros* ou *argumentos*.

Pyhton

Funções

- Conjunto de instruções onde o resultado será utilizado posteriormente.
- Assim, após ser invocado, há retorno de valor após a execução das instruções.
- Exemplos de utilização:
 - Ordenação de valores.
 - Leitura de arquivos.
 - Cálculos diversos.
- Exemplo de declaração de uma Função em pseudocódigo:

```
função <nome-função> (<nome-variavel>:<tipo-de-dado>, ...) : <tipo-de-dado>  
variaveis  
    <declaração de variáveis>  
inicio  
    <instruções>  
    <nome-função> <- <valor-retorno>  
fim
```

Python

Funções

- Palavra reservada “**def**” representa a definição da função.
- Palavra reservada “**return**”, devolve o resultado da função ao chamador.
- Declaração de uma Função em Python:

```
def soma(a, b):  
    return a+b
```

```
def modulo(a, b):  
    return a%b
```

```
print(soma(2, 40))  
print(modulo(10, 3))
```

Python

Arquivos de Módulo

- Contém as definições de um módulo
- Podem conter instruções Python, classes ou funções
- Pode ser importado para outros módulos
- O nome do arquivo é o nome do módulo:

- “`meuModuloMatematico.py`”

- Conteúdo do arquivo

```
def soma(a, b):  
    return a+b
```

```
def modulo(a, b):  
    return a%b
```



Saiba mais

- Sobre módulos, visite:
<https://docs.python.org/pt-br/3/tutorial/modules.html#standard-modules>

Python

Utilizando um Módulo

```
import meuModuloMatematico
```

```
numero_1 = 20
```

```
numero_2 = 10
```

```
resultado = soma(numero_1, numero_2)
```

```
modulo = modulo(numero_1, numero_2)
```

```
print( f"A soma de {numero_1} e {numero_2} é {resultado}" )
```

```
print( f"O modulo entre {numero_1} e {numero_2} é {modulo}" )
```



Saiba mais

- Sobre módulos, visite:
<https://docs.python.org/pt-br/3/tutorial/modules.html#standard-modules>

Pyhton

Arquivos de Módulo - Exemplos

- **Math** – fornece funções matemáticas
- **Random** – fornece geradores de números aleatórios
- **Requests** – permite fazer requisições HTTP
- **Numpy** – utilizado para manipulação de arrays
- **Panda** – utilizado em análise de dados
- **Sys** – fornece acesso a variáveis utilizadas ou mantidas pelo interpretador e a funções que interagem com o interpretador



Saiba mais

- Sobre módulos, visite:
<https://docs.python.org/pt-br/3/tutorial/modules.html#standard-modules>

Python

Classes

- Gabaritos para a construção de objetos.
- Receitas usadas para instanciar/criar objetos.
- Definem:
 - Os tipos de **atributos/propriedades** do objeto
 - Os métodos deste objeto
- Classe:
 - Membros da Classe:
 - **atributos** + **métodos**
- Implementação de características-chave de POO:
 - Encapsulação
 - Herança
 - Polimorfismo



Saiba mais

- Sobre classes, visite:
<https://www.dca.fee.unicamp.br/cursos/POOCPP/node10.html>
- Sobre classes em Python, visite:
<https://docs.python.org/pt-br/3/tutorial/classes.html>
- No W3schools, visite:
https://www.w3schools.com/python/python_classes.asp

Pyhton

Classes – Construtores, métodos e instâncias

- **Construtor**
 - Conjunto de instruções utilizadas para iniciar uma instância de uma classe
 - Podem ter argumentos que definem os valores iniciais de um objeto construído/instanciado
- **Métodos**
 - Geralmente, são nomes formados por verbos
 - Alteram o estado da classe
- **Instâncias**
 - Representa uma classe de forma concreta
 - Chamado, também, de objeto da classe



Saiba mais

- Sobre classes, visite:
<https://www.dca.fee.unicamp.br/cursos/POOCPP/node10.html>
- Sobre classes em Python, visite:
<https://docs.python.org/pt-br/3/tutorial/classes.html>
- No W3schools, visite:
https://www.w3schools.com/python/python_classes.asp

Python

Classes – Instâncias e exemplos

```
class Cao:
    def __init__(self, peso, raca):
        self.peso = peso
        self.raca = raca

    def __str__(self):
        return f'{self.raca} ({self.peso})'

meuCao = Cao (10, "poodle")
print("Meu {meuCao.raca} pesa {meuCao.peso} Kg")
print(meuCao)
```



Saiba mais

- Sobre classes e instâncias de classe, visite: <https://docs.python.org/pt-br/3/tutorial/classes.html>

Python

Classes – Métodos e exemplos

```
class Cao:
    def __init__(self, peso, raca):
        self.peso = peso
        self.raca = raca

    def __str__(self):
        return f'{self.raca} ({self.peso})

    def latir(self):
        print("Au, au!")

meuCao = Cao(10, "poodle")
meuCao.latir()
```



Saiba mais

- Sobre classes e instâncias de classe e métodos, visite: <https://docs.python.org/pt-br/3/tutorial/classes.html>

Python

Classes – Propriedades Públicas

```
class Compra:
```

```
    def __init__(self, codigo, valor):
```

```
        self.codigo = código
```

```
        self.valor = valor
```

```
        self.produtos = []
```

```
    def incluir_produtos (self, produto):
```

```
        self.produtos.append(produto)
```

```
c = Compra(20, 10.0)
```

```
c.incluir_produtos("calça")
```

```
c.incluir_produtos("camisa")
```

```
print( f"Compra {c.codigo} possui os produtos {c.produtos}" )
```



Saiba mais

- Sobre classes e instâncias de classe e propriedade, visite:
<https://docs.python.org/pt-br/3/tutorial/classes.html>

Python

Classes – Propriedades Privadas

```
class Compra:
```

```
    def __init__(self, codigo, valor):
```

```
        self.__codigo = codigo
```

```
        self.__valor = valor
```

```
    def get_codigo(self):
```

```
        return self.__codigo
```

```
    def get_valor(self):
```

```
        return self.__valor
```

```
c = Compra(20, 10.0)
```

```
print( f"Compra {c.get_codigo()} de R$ {c.get_valor()}" )
```



Saiba mais

- Sobre classes e instâncias de classe e propriedades:
<https://docs.python.org/pt-br/3/tutorial/classes.html>

Python

Classes – Herança

```
class Pessoa:
```

```
    def __init__(self, nome, idade):  
        self.nome = nome  
        self.idade = idade
```

```
class Crianca(Pessoa):
```

```
    pass
```

```
class Empregado(Pessoa):
```

```
    def __init__(self, nome, idade, salario):  
        super().__init__(self.nome, self.idade)  
        self.salario = salario
```

```
p = Pessoa("Jose", 20)
```

```
print( f"Eu me chamo {p.nome} e tenho  
      {p.idade}" )
```

```
c = Crianca("Maria", 10)
```

```
print( f"Eu me chamo {p.nome},  
      tenho {p.idade} e sou uma  
      criança" )
```

```
e = Empregado("Sula", 35,  
             1000.50)
```

```
print( f"Eu me chamo {e.nome},  
      tenho {e.idade} e recebo um  
      salário no valor de R$  
      {e.salario}" )
```



Saiba mais

- Sobre classes e instâncias de classe e herança, visite: <https://docs.python.org/pt-br/3/tutorial/classes.html>

Python

Praticando

Problemas:

1. Utilize funções em exercícios da lista de exercícios de módulos e da lista de rotinas.
2. Utilize classes em exercícios da lista de exercícios de registros.
3. Utilize funções, classes e métodos em exercícios da lista de registros.

Obs.:

Muitos dos exercícios das listas utilizadas nos exercícios do curso, permitem que você utilize uma ou mais funções, classes, métodos de classe, vetores, listas, tuplas, conjuntos, dicionários.

O limite está na avaliação da possibilidade de se utilizar estas estruturas de dados e/ou de instruções no algoritmo de solução do problema.

Python

- Considerações finais.
- Agradecimentos.



Programação em Python

Botafogo



Até a próxima!

